

Практическая работа

Загрузка Linux

Цели работы:

- изучить этапы загрузки операционных систем семейства GNU/Linux
- освоить методы анализа процесса загрузки
- получить навыки настройки параметров загрузчика, диагностики проблем и оптимизации времени старта системы.

Процесс загрузки Linux представляет собой многоэтапную последовательность, включающую инициализацию оборудования, запуск загрузчика, ядра, временной файловой системы и системы инициализации.

Этапы загрузки

1. Включение питания и инициализация оборудования

После включения питания процессор начинает выполнение кода прошивки BIOS в старых системах или UEFI в современных. Этот код выполняет проверку оборудования (POST), инициализирует устройства, а затем ищет загрузочное устройство (жёсткий диск, SSD, USB и т.д.) и передает управление загрузчику. В BIOS загрузчик считывается из первого сектора диска (MBR). В UEFI загрузчиком является файл в специальном разделе ESP (EFI System Partition), а выбор устройства осуществляется через UEFI Boot Manager.

2. Загрузчик (Bootloader)

Наиболее распространённым загрузчиком в Linux является GRUB (Grand Unified Bootloader). Его задачи:

- предоставить пользователю меню для выбора операционной системы или версии ядра
- загрузить выбранное ядро Linux в память и передать ему параметры командной строки
- загрузить в память образ `initramfs` (initial RAM filesystem), который будет использован ядром на следующем этапе.

Конфигурация GRUB хранится в файле `/boot/grub/grub.cfg` (генерируется автоматически). Пользовательские настройки (время ожидания, пункт по умолчанию, дополнительные параметры ядра и др.) задаются в `/etc/default/grub` и скриптах каталога `/etc/grub.d/`. Для обновления конфигурации после изменений используется команда **`update-grub`** (Debian/Ubuntu) или **`grub2-mkconfig`** (Red Hat/Fedora).

3. Ядро Linux

Ядро - центральный компонент операционной системы. После загрузки оно:

- инициализирует оборудование (процессоры, память, устройства).
- распаковывает `initramfs` в оперативной памяти и запускает из него процесс `/init`.
- принимает параметры командной строки, переданные от GRUB (их можно посмотреть в файле `/proc/cmdline`). Параметры влияют на поведение ядра и процесс загрузки (например, `quiet` подавляет сообщения, `splash` - графическая заставка, `ro` - монтирование корневой файловой системы только для чтения, `root=UUID=...` указывает корневую файловую систему).

4. Initramfs (временная корневая файловая система)

Initramfs — небольшой образ файловой системы, содержащий минимальный набор драйверов и утилит, необходимых для монтирования настоящей корневой файловой системы. Он загружается в память вместе с ядром и выполняет следующие задачи:

- загружает драйверы для дисков, контроллеров, файловых систем
- монтирует корневую файловую систему (указанную параметром `root=...`)
- передаёт управление процессу инициализации, расположенному в реальной корневой файловой системе (обычно `/sbin/init`).

Содержимое initramfs можно просмотреть с помощью команды ***lsinitramfs***:
lsinitramfs /boot/initrd.img-\$(uname -r).

Обновление initramfs: ***update-initramfs -u*** (Debian/Ubuntu) или ***dracut --force*** (Red Hat/Fedora).

5. Система инициализации (init)

Первым процессом, запускаемым ядром, является процесс с PID 1. В современных дистрибутивах это **systemd**.

Его основная задача - запустить все необходимые службы (сервисы) в порядке зависимостей, обеспечивая переход к целевому состоянию (**target**).

Основные цели:

- **multi-user.target** - многопользовательский режим без графики
- **graphical.target** - многопользовательский режим с графическим интерфейсом
- **rescue.target** - однопользовательский режим для восстановления.

Для анализа загрузки используется утилита **systemd-analyze**:

systemd-analyze time - общее время загрузки

systemd-analyze blame - время запуска каждого сервиса

systemd-analyze critical-chain - цепочка критических зависимостей.

systemd-analyze plot > boot.svg - график загрузки в формате SVG.

6. Логи загрузки

Сообщения ядра хранятся в кольцевом буфере и доступны через команду ***dmesg***.

Системный журнал **systemd (journal)** содержит логи всех служб и ядра. Просмотр логов:

- для текущей загрузки: ***journalctl -b***
- фильтрация по приоритету: ***journalctl -b -p err*** (только ошибки)
- по сервису: ***journalctl -b -u имя_сервиса.service***.

Практическое задание

Задание 1: Карта загрузки системы

Цель: понять последовательность этапов загрузки Linux, определить основные компоненты и проанализировать параметры, переданные ядру.

Задачи

1. Определить систему инициализации.

2. Изучить параметры загрузки ядра и объяснить их назначение.
3. Оценить время загрузки.
4. Посмотреть загруженные модули ядра.

Порядок выполнения

1. Определите, какая система инициализации используется (PID1):

```
file /sbin/init           # обычно ссылка на systemd
systemctl --version      # версия systemd
ps -p 1 -o comm=         # имя процесса с PID 1
```

2. Получите параметры командной строки, переданные ядру:

```
cat /proc/cmdline
```

Изучите вывод команды, найдите и объясните следующие параметры (если они присутствуют):

- **BOOT_IMAGE=...** - путь к образу ядра.
- **root=...** - указание корневой файловой системы (может быть задана через **UUID=** или **/dev/...**)
- **ro** - монтирование корневой файловой системы в режиме «только чтение» на начальном этапе
- **quiet** - подавление вывода сообщений ядра
- **splash** - отображение графической заставки
- **vt.handoff=...** - передача управления виртуальному терминалу.

3. Определите время загрузки системы:

```
systemd-analyze time      # или просто systemd-analyze
```

Визуализация графика загрузки:

```
systemd-analyze plot > ~/boot.svg
```

Примечание: Команда **systemd-analyze plot > ~/boot.svg** не выводит данные на экран, а сохраняет детальный график загрузки в формате SVG в файл **boot.svg** в вашем домашнем каталоге. Этот файл можно открыть в любом браузере или программе просмотра изображений. Для проверки создания файла используйте **ls -l ~/boot.svg**.

4. Список загруженных модулей ядра (первые 10)

```
lsmod | head -10
```

Вопросы для отчета:

1. Перечислите этапы загрузки в правильном порядке (от включения питания до появления приглашения входа).
2. Какой процесс запускается первым после ядра? Какова его роль?
3. Сколько времени заняла загрузка вашей системы (kernel + userspace)?
4. Запишите содержимое **/proc/cmdline**. Что означают параметры **quiet**, **splash**, **ro**? Какой **UUID** у вашей корневой файловой системы? Зачем используется **UUID** вместо имени устройства (например, **/dev/sda1**)?
5. Какие модули ядра загружены первыми? (из вывода **lsmod**)

Задание 2. Исследование загрузчика GRUB

Цель: изучить конфигурацию загрузчика GRUB, научиться находить доступные ядра и параметры загрузки.

Задачи

1. Понять структуру конфигурационного файла GRUB.
2. Определить все доступные пункты меню (включая подменю).
3. Определить настройки по умолчанию.

Порядок выполнения

1. Получите список доступных ядер в системе:

```
ls /boot/vmlinuz* | sort -V
```

2. Изучите структуру файла конфигурации GRUB

Посмотрите начало файла, чтобы понять его синтаксис

```
head -30 /boot/grub/grub.cfg
```

Или просмотрите файл постранично (выход по q)

```
less /boot/grub/grub.cfg
```

Конфигурационный файл /boot/grub/grub.cfg создаётся автоматически и его не рекомендуется редактировать вручную. Содержит множество служебных строк, в том числе условные операторы (if), которые не являются пунктами меню. Строки, объявляющие пункты меню, всегда содержат слово menuentry. Основной пункт меню (обычно первая запись) не имеет отступа, а вложенные пункты внутри подменю - имеют пробелы или табуляции в начале строки.

3. Получите список всех пунктов меню GRUB:

```
grep "menuentry" /boot/grub/grub.cfg | grep -v "if" | grep -v "export"
```

Извлеките только названия пунктов меню:

```
grep "menuentry" /boot/grub/grub.cfg | grep -v "if" | grep -v "export" | cut -d '"' -f2
```

Чтобы учесть вложенные пункты (с отступами), можно использовать более универсальный шаблон:

```
grep -E "^[[:space:]]*menuentry" /boot/grub/grub.cfg | cut -d '"' -f2
```

*[[[:space:]]** означает любое количество пробелов или табуляций перед menuentry.

4. Узнайте версию GRUB:

```
grub-install --version
```

5. Просмотрите основные настройки загрузчика:

```
cat /etc/default/grub
```

```
grep GRUB_TIMEOUT /etc/default/grub # время показа меню
```

```
grep GRUB_DEFAULT /etc/default/grub # пункт по умолчанию
```

6. Перезагрузите систему и войдите в меню GRUB (при необходимости удерживайте Shift или нажимайте Esc). Сделайте скриншот меню загрузки. Выберите пункт «Advanced options» и изучите доступные ядра.

Вопросы для отчета:

1. Сколько различных пунктов меню (включая подменю) вы обнаружили в файле grub.cfg?
1. Сколько ядер доступно для загрузки в вашей системе? Укажите их версии.
2. Какое ядро загружается по умолчанию? Как это определено в /etc/default/grub?
3. Какой параметр в файле /etc/default/grub отвечает за время отображения меню? Какое значение установлено у вас?
4. Что произойдет, если установить GRUB_TIMEOUT=0?

Задание 3. Анализ времени загрузки

Цель: научиться определять, какие службы замедляют старт системы.

Задачи

1. Получить общее время загрузки.
2. Найти самые «медленные» сервисы.
3. Построить цепочку критических зависимостей.

Порядок выполнения

1. Определите общее время загрузки (ядро + userspace):

systemd-analyze

Получите:

2. время инициализации каждого сервиса (первые 10):

systemd-analyze blame | head -10

3. критическую цепочку загрузки (самые важные зависимости):

systemd-analyze critical-chain

4. список всех сервисов и их состояние:

systemctl list-units --type=service | head -15

Проанализируйте полученные результаты, составьте таблицу из 5 сервисов с наибольшим временем запуска. Определите, какие сервисы должны загрузиться до появления сети.

Вопросы для отчета:

1. Какой сервис грузится дольше всех? Сколько времени это занимает?
2. Что означает состояние active (running) у сервиса?
3. Какой target отвечает за графический интерфейс? Как проверить, активен ли он?
4. Как переключиться в текстовый режим без перезагрузки?

Задание 4. Исследование Initramfs

Цель: понять назначение временной корневой файловой системы (initramfs).

Задачи

1. Найти файл initramfs для текущего ядра.
2. Оценить его размер и дату создания.
3. Изучить содержимое initramfs.

Порядок выполнения

1. Получить информацию о файле **initramfs**:

ls -lh /boot/initrd.img-\$(uname -r)

2. Прочитать содержимое initramfs (первые 20 строк):

lsinitramfs /boot/initrd.img-\$(uname -r) | head -20

3. Найти драйверы внутри initramfs:

lsinitramfs /boot/initrd.img-\$(uname -r) | grep -E "\.ko\$" | head -10

4. Найти скрипты инициализации внутри initramfs:

lsinitramfs /boot/initrd.img-\$(uname -r) | grep "scripts"

Вопросы для отчета:

1. Какой размер у вашего **initramfs**? (в мегабайтах)
2. Для чего нужен **initramfs**?
3. Какие драйверы (модули) присутствуют в **initramfs**? Приведите 2–3 примера.
4. Что произойдёт, если удалить **initramfs**?

Задание 5: Мониторинг процесса загрузки (логи)

Цель: научиться читать и анализировать сообщения, генерируемые при загрузке.

Задачи

1. Просмотреть логи текущей загрузки.
2. Найти сообщения ядра.
3. Обнаружить возможные ошибки.

Порядок выполнения

Получите:

1. время начала текущей загрузки:

uptime -s

who -b

2. все логи текущей загрузки (просмотр через less):

journalctl -b | less

3. только сообщения ядра:

dmesg | head -30 ***# первые 30 строк***

dmesg | grep -i "error|warn" ***# поиск ошибок/предупреждений***

4. только логи с ошибками (приоритет err и выше):

journalctl -b -p err

5. логи конкретного сервиса (например, network):

journalctl -b -u systemd-networkd

Найдите в логах момент монтирования корневой файловой системы (например, ***journalctl -b | grep "Mount.*root"***). Определите, какие устройства были обнаружены при загрузке (диски, сетевые карты). Проверьте, есть ли в логах предупреждения (warnings).

Вопросы для отчета:

1. В какое время началась текущая загрузка системы?
2. Были ли ошибки при загрузке? Если да, то какие?
3. Какой сервис, судя по логам, запустился первым? (после ядра)

Задание 6. Сравнение состояний системы

Цель: проанализировать состояние уже загруженной системы.

Задачи

1. Подсчитать загруженные модули ядра.
2. Посмотреть смонтированные файловые системы.
3. Изучить сетевые интерфейсы и запущенные процессы.

Порядок выполнения

Получите информацию о:

1. количестве загруженных модулей:

lsmod | wc -l

2. смонтированных файловых системах (только блочные устройства):

mount | grep "^/dev"

3. использовании дискового пространства:

df -h

4. сетевых интерфейсах:

ip addr show

5. количестве запущенных процессов:

ps -e --no-headers | wc -l

Вопросы для отчета:

1. Сколько модулей ядра загружено в системе?
2. Какая файловая система используется для корневого каталога (/)?
3. Какие сетевые интерфейсы активны? Есть ли у них IP-адреса?
4. Сколько процессов работает в системе? (примерно)

Задание 7. Эксперимент с уровнями выполнения (targets)

Цель: понять разницу между различными состояниями системы в **systemd**.

Задачи

1. Изучить доступные targets.
2. Узнать текущий target по умолчанию.
3. Сравнить multi-user.target и graphical.target.

Порядок выполнения

Получите:

1. список всех доступных targets:

systemctl list-unit-files --type=target

2. текущий target по умолчанию:

systemctl get-default

3. сервисы, необходимые для multi-user.target:

systemctl list-dependencies multi-user.target | head -20

4. сервисы, необходимые для graphical.target:

systemctl list-dependencies graphical.target | head -20

5. текущий активный target:

systemctl list-units --type=target --state=active

Сравните списки зависимостей для **multi-user.target** и **graphical.target**. Какие дополнительные сервисы запускаются в графическом режиме? Определите, какой сервис отвечает за менеджер входа (display manager). Например:

grep -l "display-manager" /lib/systemd/system/* или ***systemctl status display-manager***.

Обратите внимание, что переключение в multi-user.target закроет графическую сессию. Работу следует выполнять в виртуальной машине или быть готовым вернуться в графический режим через **sudo systemctl isolate graphical.target**.

Вопросы для отчета:

1. В каком target работает ваша система сейчас?
2. Какие сервисы есть в graphical.target, но отсутствуют в multi-user.target?
3. Как переключиться в текстовый режим (multi-user) без перезагрузки? Выполните команду (с предупреждением о закрытии графических приложений):

sudo systemctl isolate multi-user.target

Контрольные вопросы

1. Назовите основные этапы загрузки Linux.
2. В чём различие между BIOS и UEFI?
3. Какова роль GRUB в процессе загрузки?

4. Что такое initramfs и для чего он нужен?
5. Какая информация содержится в файле /proc/cmdline?
6. Как узнать, какая система инициализации используется?
7. Перечислите основные команды systemd-analyze.
8. Как посмотреть логи только текущей загрузки?
9. Что такое target в systemd? Приведите примеры.
10. Как добавить параметр ядра временно (при загрузке) и постоянно?

Дополнительное задание

Создание минимального собственного initramfs

Цель: самостоятельно сформировать минимальный образ initramfs, способный смонтировать корневую файловую систему и передать управление systemd, выполнить загрузку с использованием этого образа.

Задачи

1. Изучить структуру стандартного initramfs.
2. Создать каталог для сборки и наполнить его необходимыми компонентами.
3. Написать скрипт инициализации (init), который выполнит подготовку окружения и переключится на реальный корень.
4. Упаковать содержимое в cpio-архив (сжатый gzip).
5. Настроить GRUB для загрузки с новым initramfs.
6. Протестировать загрузку, проанализировать возможные ошибки.
7. Сравнить размер и состав самодельного initramfs со стандартным.

Порядок выполнения

Работу рекомендуется выполнять в виртуальной машине, чтобы в случае ошибки можно было легко восстановить систему.

1. Создайте рабочий каталог, например:
`mkdir ~/myinitramfs && cd ~/myinitramfs`
2. Просмотрите содержимое текущего **initramfs**:
`mkdir /tmp/initrd_content && cd /tmp/initrd_content`
`zcat /boot/initrd.img-$(uname -r) | cpio -id`
`ls -l`

Обратите внимание на структуру каталогов (bin, dev, etc, lib, lib64, proc, sbin, sys, usr, init и т.д.) и наличие утилит (например, busybox, mount, switch_root). Изучите скрипт /init (или ссылку на него).

3. Вернитесь в рабочий каталог ~/myinitramfs и создайте необходимые подкаталоги:
`mkdir -p bin dev etc lib lib64 mnt proc sbin sys usr/bin usr/sbin`
4. Добавьте исполняемые файлы

Проще всего использовать статически скомпилированный BusyBox, который включает большинство нужных утилит. Если BusyBox не установлен, установите его:

`sudo apt install busybox-static` # для Debian/Ubuntu

Скопируйте статический BusyBox в каталог /bin:
`cp /bin/busybox bin/`

Затем создайте символичные ссылки на него для основных команд (это можно сделать позже внутри init-скрипта, но удобнее сразу):

```
cd bin  
for applet in mount umount switch_root sh echo cat ls modprobe; do  
ln -s busybox $applet  
done  
cd ..
```

Примечания:

- switch_root - обязательная утилита для переключения корня.
- если вы решите использовать динамически скомпилированные утилиты, необходимо скопировать соответствующие библиотеки (обычно из /lib и /lib64).

5. Скопируйте необходимые модули для доступа к диску и файловой системе (например, для SATA-контроллера и ext4). Выясните, какие модули используются вашей системой:

```
lsmod | grep -E "^(ahci|libata|ext4)" # пример
```

Модули находятся в /lib/modules/\$(uname -r)/. Скопируйте нужные каталоги и зависимости:

```
mkdir -p lib/modules/$(uname -r)  
cp -a /lib/modules/$(uname -r)/kernel/drivers/ata lib/modules/$(uname -r)/kernel/drivers/  
cp -a /lib/modules/$(uname -r)/kernel/fs/ext4 lib/modules/$(uname -r)/kernel/fs/
```

Также скопируйте modules.dep и modules.alias:

```
cp /lib/modules/$(uname -r)/modules.{dep,alias} lib/modules/$(uname -r)/
```

6. Создайте файл init в корне рабочего каталога (~/.myinitramfs/init) со следующим содержимым (адаптируйте под свою файловую систему):

```
#!/bin/sh  
# Монтирование необходимых псевдо-ФС  
mount -t proc proc /proc  
mount -t sysfs sysfs /sys  
mount -t devtmpfs devtmpfs /dev  
# Загрузка модулей (если не встроены в ядро)  
modprobe ahci  
modprobe ext4  
# Поиск корневого устройства (можно по UUID или метке)  
# Здесь для упрощения будем использовать /dev/sda2 — замените на свой  
корневой раздел!  
rootdev="/dev/sda2"  
# Монтирование корневой ФС  
mount -o ro $rootdev /mnt  
# Очистка и переключение корня  
umount /proc /sys /dev  
exec switch_root /mnt /sbin/init
```

7. Сделайте скрипт исполняемым: **chmod +x init**.

Важно правильно указать устройство для корневого раздела. Его можно узнать, выполнив в работающей системе:

```
mount | grep " / "
```

8. Убедитесь, что находитесь в каталоге ~/myinitramfs, затем выполните:

```
find . | cpio -o -H newc | gzip > ~/myinitrd.img
```

Полученный файл ~/myinitrd.img - ваш новый initramfs.

9. Настройка GRUB для использования нового initramfs

Скопируйте образ в /boot (возможно, потребуется sudo):

```
sudo cp ~/myinitrd.img /boot/
```

Отредактируйте конфигурацию GRUB, чтобы добавить временный пункт меню с вашим initramfs. Самый безопасный способ - при загрузке вручную отредактировать строку параметров. Для этого:

- перезагрузитесь
- в меню GRUB выберите нужную запись и нажмите “e”.
- найдите строку, начинающуюся с linux (параметры ядра) и строку initrd
- замените путь к initrd на /boot/myinitrd.img (или просто myinitrd.img, если он в корне /boot).
- нажмите Ctrl+X или F10 для загрузки.

Если всё сделано правильно, система должна загрузиться с вашим минимальным initramfs.

10. Отладка

Если загрузка не удалась, добавьте в скрипт init отладочные вызовы оболочки. Поскольку в самодельном initramfs нет встроенных механизмов отладки (как rd.shell в стандартных образах), вы можете вставить перед ключевыми действиями (например, перед монтированием корня) следующий код:

```
echo "Нажмите Enter для продолжения или введите exit для оболочки"
```

```
sh
```

При достижении этой строки выполнение приостановится и запустится командная оболочка. В ней вы сможете вручную:

- проверить наличие устройств в /dev
- загрузить модули вручную (modprobe)
- попытаться смонтировать корневую файловую систему и проанализировать ошибки
- просмотреть содержимое каталогов.

После ввода exit скрипт продолжит работу. Необходимо удалить эти отладочные строки из финальной версии init.

11. После успешной загрузки сравните размер вашего образа со стандартным:

```
ls -lh /boot/initrd.img-$(uname -r) /boot/myinitrd.img
```

Изучите, чего не хватает в вашем образе по сравнению со стандартным.

Вопросы для отчета

1. Какие каталоги и файлы обязательно должны присутствовать в initramfs? Почему?
2. Как работает скрипт init? Опишите каждую команду и её назначение.

3. Почему после монтирования корневой файловой системы необходимо использовать `switch_root`, а не просто `exec /sbin/init`?
4. Какие трудности возникли при создании собственного `initramfs`? Как вы их преодолели?
5. Насколько ваш образ меньше стандартного? За счёт чего удалось уменьшить размер?
6. Что произойдет, если не скопировать модули ядра для дискового контроллера?
7. Как можно модифицировать скрипт `init`, чтобы он определял корневое устройство по UUID (а не по жестко заданному `/dev/sda2`)?

Критерии успешного выполнения

- Система загружается с использованием самодельного `initramfs` без ошибок.
- Предоставлен скрипт `init` и объяснена его логика.
- Проведен анализ размера и состава.
- Даны ответы на вопросы для отчета.